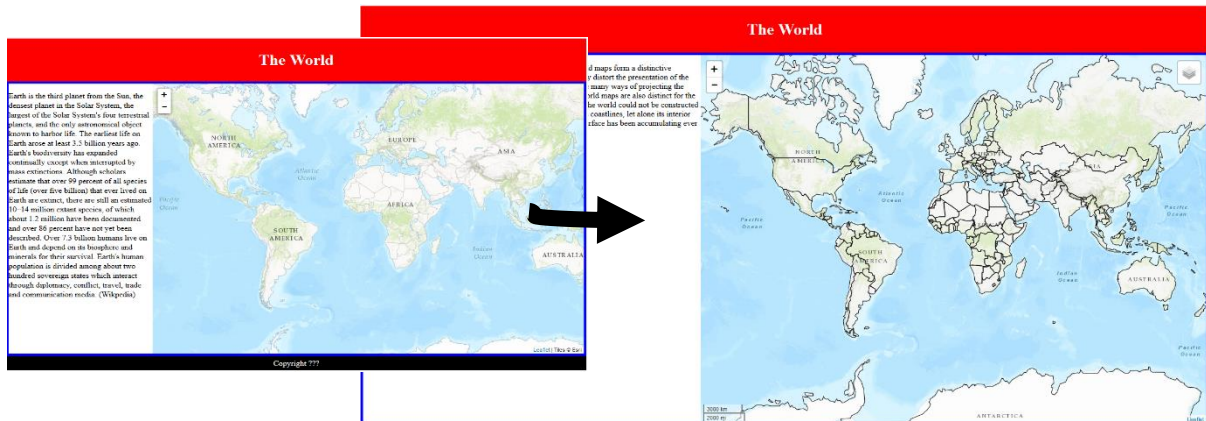


Internet-GIS WS22/23, Task 4:

Using your result from exercise 3 (html page including a Leaflet base map) your next task is to parse files in different formats and add further layers to the map.



Task 4.0: Upload files to WebGIS Web Server

1. This step will be required for testing of server-side functionality (execution of php scripts) in the context of the php exercise and the final exercise. You can skip reading this paragraph if you want to work locally for this exercise.
2. WebServer 130.75.188.102 is accessible worldwide using http(s), from LUH VPN using SSH.
3. Develop your code locally. You can test the results of this exercise locally. In case you already want to try out the WebGIS server, follow the hints on the final exercise slides/handout for a description of the steps necessary to upload files to your group folder (scp FILENAME grpX@130.75.188.102:). Then, to open the website stored on the server, open "http://130.75.188.102/grpX/FILENAME" in a Browser. Ignore the security warning and provide your group credentials.

Task 4.1: Parse XML file

1. Implement a parser for the example XML String contained in the variable *countries_xml*, defined in the .js file <https://gis.ikg.uni-hannover.de/public/countries.js>. Import this JavaScript file in the header of your html document. You can access this variable anywhere in your code by its name. Alternatively, you can directly access the XML file <https://gis.ikg.uni-hannover.de/public/countries.xml>.
2. Let the XML parser iterate through all objects of type "Feature", e.g. by using jQuery command `$(countries_xml).find("Feature").each(function() {...})`.
3. For each feature, create a Polygon Layer using the geometry information stored in Property "Geometry". You can parse an array object from the String using function `JSON.parse(string)`. Please note that coordinates are given in the XML file in order [lat, lon], i.e. compatible with Leaflet datatype LatLon.

Task 4.2: Parse GeoJSON file

1. Implement a parser for the example JSON String contained in the variable *countries_json*, defined in the .js file <https://gis.ikg.uni-hannover.de/public/countries.js>. Import this JavaScript file in the header of your html document. You can access this variable anywhere in your code by its name. Alternatively, you can directly access the GeoJSON file <https://gis.ikg.uni-hannover.de/public/countries.geojson>.
2. For each feature, create a GeoJSONLayer (<https://leafletjs.com/reference.html#geojson>) directly. Please note that coordinates in GeoJSON are given in order [lon, lat], i.e. not directly compatible with Leaflet datatype LatLon.

Task 4.3: Map styling

1. Create different styles for Layers added in 4.1 or 4.2 and assign them automatically during parsing (e.g. depending on properties of the features).
2. Create a style for Layers displayed during a mouseover event. Add a mouseover-EventHandler to all Layers during parsing (e.g. highlight a layer when the mouse hovers over it). Make sure the previous styling is restored when the mouse no longer hovers over the layer (by additionally defining a mouseout event).

Please upload your results to Stud.IP folder **Documents > Results of Exercise 5** until **29.11.2022**.